REST PDF V2 – Spring Boot REST Interview Questions (1–2 Years Experience)

Full Content from Question 1 to Question 30

1) What is REST?

Interview Question
What is REST and why is it used in Spring Boot applications?

Detailed Answer
REST stands for Representational State Transfer. It is an architectural style used to build web services that communicate over HTTP. In Spring Boot, REST is commonly used to expose APIs so frontend apps, mobile apps, or other systems can interact with backend services.

REST is based on:
• Resources (like users, products, orders)
• Each resource is identified by a URI
• Operations are performed using HTTP methods:
- GET → fetch data
- POST → create data
- PUT → update data
- DELETE → delete data

Example:
• GET /users/1 → get user with ID 1
• POST /users → create a new user

REST APIs are:
• Stateless
• Lightweight
• Easy to integrate with frontend frameworks like React/Angular
• Ideal for microservices and modern applications

Simple Explanation
REST means: “Backend exposes data as URLs, and clients use HTTP methods to interact with it.”

Example
```
@RestController
@RequestMapping("/users")
public class UserController {
```

```
    @GetMapping("/{id}")
    public String getUser(@PathVariable Long id) {
        return "User id: " + id;
    }
}
```

Interviewer Cross-Question
Q: What does stateless mean in REST?
A: Each request should contain all necessary information. The server should not depend on previous requests to understand the current one.

2) What is @RestController?

Interview Question
What is @RestController in Spring Boot?

Detailed Answer
@RestController is a Spring annotation used to create REST APIs. It is a combination of:
- @Controller
- @ResponseBody

When we use @RestController, the return value of methods is written directly into the HTTP response body as JSON, XML, or plain text instead of returning a view name.

This is especially useful in REST APIs where we usually return:
- Java objects → converted to JSON
- Strings
- DTOs
- Lists

Spring Boot automatically uses Jackson to convert Java objects into JSON.

Simple Explanation
@RestController means: “This class handles API requests and returns data directly, not JSP/HTML page names.”

Example
```
@RestController
public class HelloController {
    @GetMapping("/hello")
    public String hello() {
        return "Hello REST API";
```

```
    }
}
```

Interviewer Cross-Question
Q: What happens if I use @Controller instead?
A: Then Spring assumes the return value is a view name unless @ResponseBody is added.

3) Difference between @Controller and @RestController

Interview Question
What is the difference between @Controller and @RestController?

Detailed Answer
Both are used in Spring MVC, but their purpose differs.

@Controller
- Used mainly for traditional web applications
- Returns view names (like JSP, Thymeleaf pages)
- If you want raw data response, you must use @ResponseBody

@RestController
- Used for REST APIs
- Returns data directly in the response body
- No need to add @ResponseBody for every method

Key difference:
- @Controller → View rendering
- @RestController → JSON/data response

Simple Explanation
- @Controller = for HTML page
- @RestController = for API response

Example
```
@Controller
public class PageController {
    @GetMapping("/home")
    public String home() {
        return "home"; // returns view name
    }
}
```

```
@RestController
public class ApiController {
   @GetMapping("/api/home")
   public String home() {
      return "home"; // returns response body
   }
}
```

Interviewer Cross-Question
Q: Is @RestController internally equal to @Controller + @ResponseBody?
A: Yes, exactly.

4) Explain @GetMapping, @PostMapping, @PutMapping, @DeleteMapping

Interview Question
What are @GetMapping, @PostMapping, @PutMapping, and @DeleteMapping in Spring REST?

Detailed Answer
These annotations are shortcut annotations for mapping HTTP methods in REST APIs. They are specialized forms of @RequestMapping.

1. @GetMapping
Used to fetch/read data.

2. @PostMapping
Used to create new data.

3. @PutMapping
Used to update existing data completely.

4. @DeleteMapping
Used to remove data.

These annotations make the code more readable and align clearly with REST semantics.

Simple Explanation
Each annotation maps to an HTTP action:
- GET = Read
- POST = Create
- PUT = Update
- DELETE = Remove

Example

```
@RestController
@RequestMapping("/employees")
public class EmployeeController {

   @GetMapping("/{id}")
   public String getEmployee(@PathVariable Long id) {
      return "Get employee " + id;
   }

   @PostMapping
   public String createEmployee() {
      return "Employee created";
   }

   @PutMapping("/{id}")
   public String updateEmployee(@PathVariable Long id) {
      return "Employee updated " + id;
   }

   @DeleteMapping("/{id}")
   public String deleteEmployee(@PathVariable Long id) {
      return "Employee deleted " + id;
   }
}
```

Interviewer Cross-Question

Q: What is the difference between PUT and PATCH?
A: PUT = full update, PATCH = partial update.

5) What is @RequestBody?

Interview Question

What is @RequestBody in Spring Boot REST API?

Detailed Answer

@RequestBody is used to bind the HTTP request body (usually JSON) to a Java object.

When a client sends JSON data in a POST or PUT request, Spring uses Jackson to convert that JSON into a Java object automatically.

This is commonly used for:
• Create APIs
• Update APIs
• Request DTOs

Without @RequestBody, Spring will not know to deserialize the incoming JSON payload into the method parameter.

Simple Explanation
@RequestBody means: “Take JSON from request body and convert it into Java object.”

Example

```
@RestController
@RequestMapping("/users")
public class UserController {

   @PostMapping
   public String createUser(@RequestBody User user) {
      return "Created user: " + user.getName();
   }
}

public class User {
   private String name;
   private String email;
   // getters and setters
}
```

Interviewer Cross-Question
Q: Which library converts JSON to Java object?
A: By default, Spring Boot uses Jackson.

6) What is @PathVariable?

Interview Question
What is @PathVariable in Spring REST?

Detailed Answer
@PathVariable is used to extract values from the URI path and bind them to method parameters.

It is commonly used when the value is part of the resource identifier.

Example:
• /users/10
Here 10 is part of the URL path, so it can be captured using @PathVariable.

It is ideal for:
• Resource IDs
• Nested resources
• Clean RESTful URLs

Simple Explanation
@PathVariable means: “Read value from URL path.”

Example
```
@GetMapping("/users/{id}")
public String getUser(@PathVariable Long id) {
   return "User id: " + id;
}
```

Interviewer Cross-Question
Q: Can variable name differ from path name?
A: Yes.
```
@GetMapping("/users/{userId}")
public String getUser(@PathVariable("userId") Long id) { return "User id: " + id; }
```

7) What is @RequestParam?

Interview Question
What is @RequestParam and how is it different from @PathVariable?

Detailed Answer
@RequestParam is used to read values from query parameters in the URL.

Example:
• /users?page=1&size=10

It is commonly used for:
• Filtering
• Searching
• Pagination
• Sorting
• Optional values

Difference:
• @PathVariable → value from URL path
• @RequestParam → value from query string

Simple Explanation
@RequestParam means: “Read value after ? in the URL.”

Example
```
@GetMapping("/users")
public String getUsers(@RequestParam int page, @RequestParam int size) {
   return "Page: " + page + ", Size: " + size;
}
```

Interviewer Cross-Question
Q: Can @RequestParam be optional?
A: Yes.
```
@GetMapping("/search")
public String search(@RequestParam(required = false) String keyword) { return keyword == null
? "No keyword" : keyword; }
```

8) What is ResponseEntity?

Interview Question
Why do we use ResponseEntity in Spring REST APIs?

Detailed Answer
ResponseEntity is used to return:
• Response body
• HTTP status code
• Headers

It gives full control over the HTTP response.

Without ResponseEntity, Spring automatically returns status 200 OK in many cases. But in real APIs, we often need:
• 201 CREATED after POST
• 404 NOT FOUND when resource missing
• 400 BAD REQUEST
• Custom headers

So ResponseEntity is the preferred approach for production-grade REST APIs.

Simple Explanation
ResponseEntity means: “Return data + status code + headers in a controlled way.”

Example
```
@GetMapping("/users/{id}")
public ResponseEntity<String> getUser(@PathVariable Long id) {
   if (id == 1) {
      return ResponseEntity.ok("User found");
   }
   return ResponseEntity.notFound().build();
}
```

Interviewer Cross-Question
Q: Why is ResponseEntity better than returning plain object?
A: It allows explicit control over HTTP status and headers, which is important in REST APIs.

9) Explain HTTP Status Codes in REST API

Interview Question
What are the important HTTP status codes used in Spring REST APIs?

Detailed Answer
HTTP status codes tell the client whether the request succeeded or failed.

Common 2xx (Success)
• 200 OK → successful GET/PUT
• 201 CREATED → resource created
• 204 NO CONTENT → success but no response body

Common 4xx (Client Error)
• 400 BAD REQUEST → invalid request
• 401 UNAUTHORIZED → authentication required
• 403 FORBIDDEN → authenticated but no access
• 404 NOT FOUND → resource not found

Common 5xx (Server Error)
• 500 INTERNAL SERVER ERROR → unexpected backend error

Using proper status codes makes APIs more professional and easier for frontend systems to consume.

Simple Explanation
Status code = “What happened to the request?”

Example
```
@PostMapping("/users")
public ResponseEntity<String> createUser() {
   return ResponseEntity.status(HttpStatus.CREATED).body("User created");
}
```

Interviewer Cross-Question
Q: What should you return after successful delete?
A: Usually 204 NO CONTENT or 200 OK depending on API design.

10) What is @RequestMapping?

Interview Question
What is @RequestMapping and how is it different from @GetMapping?

Detailed Answer
@RequestMapping is a generic annotation used to map web requests to controller methods or classes.

It can be used:
- At class level → common base path
- At method level → specific endpoint
- For any HTTP method (GET, POST, PUT, DELETE, etc.)

@GetMapping, @PostMapping, etc. are more specific and cleaner alternatives.

Difference:
- @RequestMapping = generic
- @GetMapping = GET-specific shortcut
- @PostMapping = POST-specific shortcut

Simple Explanation
@RequestMapping is the “base mapping annotation.” The others are cleaner specialized versions.

Example
```
@RestController
@RequestMapping("/api/users")
public class UserController {
```

```
    @RequestMapping(value = "/all", method = RequestMethod.GET)
    public String getAllUsers() {
        return "All users";
    }
}

// Preferred modern style:
@GetMapping("/api/users/all")
public String getAllUsers() {
    return "All users";
}
```

Interviewer Cross-Question
Q: Where do we commonly use @RequestMapping now?
A: Mostly at class level for base path.

11) DTO vs Entity

Interview Question
What is the difference between DTO and Entity in Spring REST APIs?

Detailed Answer
Entity
An Entity is a class mapped to the database using JPA annotations like @Entity. It represents the database table.

DTO (Data Transfer Object)
A DTO is used to transfer data between layers, especially between:
• Controller ↔ Client
• Service ↔ External systems

DTOs help:
• Hide sensitive fields
• Avoid exposing internal DB structure
• Customize request/response payloads
• Reduce unnecessary data transfer

Best practice:
• Use Entity for persistence
• Use DTO for API input/output

Simple Explanation

- Entity = DB model
- DTO = API model

Example

```
@Entity
public class UserEntity {
   @Id
   private Long id;
   private String name;
   private String password;
}

public class UserResponseDto {
   private Long id;
   private String name;
}
```

Interviewer Cross-Question

Q: Why should we not return entity directly?
A: It may expose sensitive/internal fields and tightly couple API to DB structure.

12) Why not expose Entity directly?

Interview Question

Why is exposing JPA entities directly in REST APIs considered a bad practice?

Detailed Answer

Returning entities directly from controller is considered risky because:

1. Security risk

Sensitive fields like:

- password
- internal flags
- audit columns

may be exposed accidentally.

2. Tight coupling

If DB schema changes, API response may also change unintentionally.

3. Lazy loading issues

JPA relationships (@OneToMany, @ManyToOne) may cause:

• LazyInitializationException
• Infinite recursion
• Performance issues

4. Overfetching
Entity may contain more data than needed.

Best practice:
Use DTOs and mapping logic (manual or MapStruct/ModelMapper).

Simple Explanation
Never return entity directly in serious projects because:
• It leaks internal structure
• It can break APIs
• It can cause serialization problems

Example
```
@GetMapping("/{id}")
public UserResponseDto getUser(@PathVariable Long id) {
   UserEntity user = userService.getById(id);
   return new UserResponseDto(user.getId(), user.getName());
}
```

Interviewer Cross-Question
Q: What issue happens with bi-directional entity relationships?
A: Infinite recursion during JSON serialization.

13) What is @JsonIgnore? (Jackson basics)

Interview Question
What is @JsonIgnore and when do we use it in Spring REST APIs?

Detailed Answer
@JsonIgnore is a Jackson annotation used to exclude a field from JSON serialization and deserialization.

It is useful when:
• You don’t want to expose sensitive fields like password
• You want to avoid circular references
• You want to hide internal fields

Example:
If a user entity contains password, we should not return it in API response.

Simple Explanation
@JsonIgnore means: “Do not include this field in JSON response.”

Example
```
public class UserResponse {
   private Long id;
   private String name;

   @JsonIgnore
   private String password;
}
```

Interviewer Cross-Question
Q: Is @JsonIgnore a full solution instead of DTO?
A: No. DTO is still the better design. @JsonIgnore is helpful but not a replacement for proper API modeling.

14) What is @Valid in REST APIs?

Interview Question
What is @Valid and how is it used in Spring Boot REST APIs?

Detailed Answer
@Valid is used to trigger Bean Validation on request objects.

When a client sends JSON data, Spring converts it into a DTO. If @Valid is applied, Spring validates the object based on annotations like:
- @NotNull
- @NotBlank
- @Size
- @Email
- @Min
- @Max

If validation fails, Spring throws an exception (MethodArgumentNotValidException), which can be handled globally.

This improves:
- Input quality

• API robustness
• Cleaner controller logic

Simple Explanation
@Valid means: “Validate incoming request object before processing.”

Example

```
public class UserRequestDto {
   @NotBlank(message = "Name is required")
   private String name;

   @Email(message = "Invalid email")
   private String email;

   // getters and setters
}

@PostMapping("/users")
public ResponseEntity<String> createUser(@Valid @RequestBody UserRequestDto dto) {
   return ResponseEntity.ok("User created");
}
```

Interviewer Cross-Question
Q: Which dependency is needed for validation in Spring Boot?
A: Usually spring-boot-starter-validation.

15) What is @ControllerAdvice?

Interview Question
What is @ControllerAdvice in Spring Boot?

Detailed Answer
@ControllerAdvice is used for global exception handling, global model attributes, and global binder configuration across multiple controllers.

In REST APIs, it is most commonly used with @ExceptionHandler to centralize exception handling instead of writing try-catch in every controller.

Benefits:
• Cleaner controllers
• Reusable exception handling

• Consistent error responses
• Easier maintenance

Simple Explanation
@ControllerAdvice means: “Handle common controller-level logic globally.”

Example

```
@ControllerAdvice
public class GlobalExceptionHandler {

   @ExceptionHandler(Exception.class)
   public ResponseEntity<String> handleException(Exception ex) {
      return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Something went wrong: " + ex.getMessage());
   }
}
```

Interviewer Cross-Question
Q: What is the REST-specific version of @ControllerAdvice?
A: @RestControllerAdvice.

16) Global Exception Handling in REST APIs

Interview Question
How do you implement global exception handling in Spring REST APIs?

Detailed Answer
Global exception handling is usually implemented using:
• @ControllerAdvice or @RestControllerAdvice
• @ExceptionHandler

Instead of handling exceptions in each controller, we define a central class that catches specific exceptions and returns a structured response.

This is important for:
• Validation errors
• Business exceptions
• Resource not found
• Generic system errors

A good production API returns:
• timestamp

• status code
• error message
• path (optional)

Simple Explanation
Global exception handling = “one place to manage all API errors.”

Example

```
@RestControllerAdvice
public class GlobalExceptionHandler {

   @ExceptionHandler(ResourceNotFoundException.class)
   public ResponseEntity<String> handleNotFound(ResourceNotFoundException ex) {
      return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ex.getMessage());
   }

   @ExceptionHandler(MethodArgumentNotValidException.class)
   public ResponseEntity<String> handleValidation(MethodArgumentNotValidException ex) {
      return ResponseEntity.status(HttpStatus.BAD_REQUEST).body("Validation failed");
   }
}
```

Interviewer Cross-Question
Q: Why is this better than try-catch in every controller?
A: It keeps controller code clean, avoids duplication, and ensures consistent error responses.

17) API Versioning

Interview Question
What is API versioning and how can it be implemented in Spring Boot?

Detailed Answer
API versioning means maintaining multiple versions of an API so existing clients don’t break when the API evolves.

Common approaches:
1. URI versioning
  - /api/v1/users
  - /api/v2/users

2. Request parameter versioning
  - /users?version=1

3. Header versioning
  - X-API-VERSION: 1

4. Media type versioning
  - Accept: application/vnd.company.app-v1+json

Most common in interviews/projects:
• URI versioning because it is simple and readable

Simple Explanation
Versioning means: “Support old API and new API together without breaking clients.”

Example

```
@RestController
@RequestMapping("/api/v1/users")
public class UserV1Controller {
   @GetMapping("/{id}")
   public String getUserV1(@PathVariable Long id) {
      return "User V1";
   }
}

@RestController
@RequestMapping("/api/v2/users")
public class UserV2Controller {
   @GetMapping("/{id}")
   public String getUserV2(@PathVariable Long id) {
      return "User V2 with extra fields";
   }
}
```

Interviewer Cross-Question
Q: Which versioning strategy is easiest and most commonly used?
A: URI versioning.

18) Idempotent Methods

Interview Question
What are idempotent HTTP methods? Explain with examples.

Detailed Answer
An HTTP method is idempotent if making the same request multiple times produces the same effect on the server.

Idempotent methods:
• GET
• PUT
• DELETE
• HEAD
• OPTIONS

Not idempotent:
• POST (usually)

Examples:
• GET /users/1 → reading multiple times does not change state
• PUT /users/1 with same data → same final state
• DELETE /users/1 repeatedly → resource remains deleted

Why important?
It helps in:
• Retry mechanisms
• Distributed systems
• Network failures
• API design clarity

Simple Explanation
Idempotent means: “Calling the same API again and again gives the same final effect.”

Example
```
@PutMapping("/users/{id}")
public ResponseEntity<String> updateUser(@PathVariable Long id, @RequestBody
UserRequestDto dto) {
   return ResponseEntity.ok("User updated");
}
```

Interviewer Cross-Question
Q: Why is POST usually not idempotent?
A: Because calling POST multiple times may create multiple new records.

19) Pagination Basics

Interview Question
How do you implement pagination in Spring Boot REST APIs?

Detailed Answer
Pagination is used when the dataset is large and we don't want to return all records at once.

In Spring Data JPA, pagination is commonly implemented using:
• Pageable
• Page<T>

Clients usually pass:
• page
• size
• sort

Benefits:
• Better performance
• Lower memory usage
• Faster frontend rendering
• Better user experience

Simple Explanation
Pagination means: "Return data in chunks instead of returning everything."

Example
```
public interface UserRepository extends JpaRepository<UserEntity, Long> {
}

@GetMapping("/users")
public Page<UserEntity> getUsers(Pageable pageable) {
   return userRepository.findAll(pageable);
}

// Sample Request:
// GET /users?page=0&size=5&sort=name,asc
```

Interviewer Cross-Question
Q: What is the difference between Page and Slice?
A: Page includes total count and pagination metadata; Slice is lighter and only tells whether next page exists.

20) CORS Basics

Interview Question
What is CORS and why is it important in Spring REST APIs?

Detailed Answer
CORS stands for Cross-Origin Resource Sharing.

Browsers block requests from one origin to another by default for security reasons.

Example:
• Frontend: http://localhost:3000
• Backend: http://localhost:8080

These are different origins, so the browser may block the request unless the backend explicitly allows it.

In Spring Boot, CORS can be configured:
• At method level
• At controller level
• Globally

This is very important in:
• React + Spring Boot
• Angular + Spring Boot
• Microservice UIs

Simple Explanation
CORS means: “Allow frontend from another port/domain to call backend API.”

Example

```
@CrossOrigin(origins = "http://localhost:3000")
@RestController
@RequestMapping("/users")
public class UserController {

   @GetMapping
   public String getUsers() {
      return "Users list";
   }
}
```

Interviewer Cross-Question
Q: Does CORS matter in Postman?
A: No. CORS is enforced by browsers, not Postman.

21) What is @RequestHeader?

Interview Question
What is @RequestHeader in Spring Boot REST API?

Detailed Answer
@RequestHeader is used to read the value of an HTTP request header and bind it to a method parameter in a controller.

In REST APIs, headers are commonly used to send:
• Authorization tokens
• Content type
• API version
• Correlation IDs / Trace IDs
• Custom client information

It can also be used for:
• Optional headers
• Default values
• Reading all headers as a Map

This annotation is very useful in real-world applications because many important request details are passed through headers rather than the URL or request body.

Simple Explanation
@RequestHeader means: “Read value from HTTP request header.”

Example

```
@RestController
@RequestMapping("/api/users")
public class UserController {

   @GetMapping
   public String getUsers(@RequestHeader("Authorization") String token) {
      return "Token received: " + token;
   }
}
```

Interviewer Cross-Question
Q: Can @RequestHeader be optional?
A: Yes.

```
@GetMapping("/info")
```

public String getInfo(@RequestHeader(value = "client-id", required = false) String clientId) {
return clientId == null ? "No client id" : clientId; }

22) Difference between @PathVariable and @RequestParam

Interview Question
What is the difference between @PathVariable and @RequestParam in Spring REST APIs?

Detailed Answer
Both @PathVariable and @RequestParam are used to read values from the request URL, but they are used in different situations.

@PathVariable
• Used to extract values from the URL path
• Example: /users/10
• Usually represents a unique resource identifier

@RequestParam
• Used to extract values from query parameters
• Example: /users?page=1&size=10
• Commonly used for filtering, pagination, sorting, and optional inputs

Best practice:
• Use @PathVariable for resource identity
• Use @RequestParam for resource filtering or optional data

Simple Explanation
• @PathVariable = value from URL path
• @RequestParam = value after ? in URL

Example

```
@RestController
@RequestMapping("/users")
public class UserController {

  @GetMapping("/{id}")
  public String getUserById(@PathVariable Long id) {
    return "User id: " + id;
  }

  @GetMapping
  public String getUsers(@RequestParam int page, @RequestParam int size) {
```

```
        return "Page: " + page + ", Size: " + size;
    }
}
```

Interviewer Cross-Question
Q: Which one is better for pagination and sorting?
A: @RequestParam, because pagination and sorting are optional query-based parameters.

23) What is @ResponseStatus?

Interview Question
What is @ResponseStatus in Spring Boot REST APIs?

Detailed Answer
@ResponseStatus is used to set the HTTP status code for a controller method or for a custom exception class.

In REST APIs, sometimes we want to return a specific HTTP status code without using ResponseEntity.

For example:
- 201 CREATED after successful resource creation
- 204 NO CONTENT after successful delete
- 404 NOT FOUND for custom exceptions

@ResponseStatus is a simpler alternative when:
- You only need to return a fixed status code
- You do not need custom headers
- You do not need dynamic response handling like ResponseEntity

Simple Explanation
@ResponseStatus means: “Return this HTTP status code automatically.”

Example
```
@RestController
@RequestMapping("/users")
public class UserController {

    @ResponseStatus(HttpStatus.CREATED)
    @PostMapping
    public String createUser() {
        return "User created";
```

```
    }
}
```

Interviewer Cross-Question
Q: What is the difference between @ResponseStatus and ResponseEntity?
A: @ResponseStatus = simple and fixed status, ResponseEntity = full control over body, status, and headers.

24) What is @PatchMapping? Difference between PUT and PATCH

Interview Question
What is @PatchMapping and what is the difference between PUT and PATCH?

Detailed Answer
@PatchMapping is used to handle HTTP PATCH requests in Spring REST APIs.

PATCH is used for partial updates, meaning only specific fields of a resource are updated.

Difference between PUT and PATCH:

PUT
- Used for full update
- Usually replaces the entire resource
- Client sends the complete updated object

PATCH
- Used for partial update
- Only updates selected fields
- Client sends only the fields that need to change

This is a very common REST design question in interviews and real projects.

Simple Explanation
- PUT = full update
- PATCH = partial update

Example
```
@RestController
@RequestMapping("/users")
public class UserController {

    @PutMapping("/{id}")
```

```
    public String updateUser(@PathVariable Long id, @RequestBody UserRequestDto dto) {
        return "User fully updated: " + id;
    }

    @PatchMapping("/{id}")
    public String updateUserEmail(@PathVariable Long id, @RequestBody Map<String, Object>
updates) {
        return "User partially updated: " + id;
    }
}
```

Interviewer Cross-Question
Q: Which one is more suitable when updating only one field like email?
A: PATCH, because it is meant for partial updates.

25) What are REST API Best Practices?

Interview Question
What best practices should we follow while designing REST APIs in Spring Boot?

Detailed Answer
Designing a good REST API is not just about writing endpoints. It is about making APIs consistent, readable, scalable, and easy for clients to use.

Important REST API Best Practices:
1. Use nouns, not verbs, in URLs
   Good: /users, /orders
   Bad: /getUsers, /createOrder

2. Use proper HTTP methods
   - GET → read
   - POST → create
   - PUT → full update
   - PATCH → partial update
   - DELETE → remove

3. Use proper HTTP status codes
4. Use DTOs instead of exposing entities directly
5. Validate request data using @Valid
6. Use global exception handling
7. Keep response format consistent

8. Use pagination for large datasets
9. Version APIs when needed
10. Keep URLs clean and meaningful

Simple Explanation
REST API best practices mean: “Design APIs that are clean, consistent, and easy to use.”

Example
Good REST design:
GET /users
GET /users/10
POST /users
PUT /users/10
PATCH /users/10
DELETE /users/10

Interviewer Cross-Question
Q: Why should we avoid URLs like /getUsers or /deleteUser?id=1?
A: Because REST APIs should be resource-oriented, and HTTP methods should represent the action.

26) What is a Standard API Response Structure?

Interview Question
Why do we use a standard API response structure in Spring Boot REST APIs?

Detailed Answer
A standard API response structure means returning all API responses in a consistent format instead of returning raw objects or plain strings.

In real-world projects, teams often use a common response wrapper so that frontend applications can handle responses consistently.

Benefits:
• Consistent response format
• Easier frontend integration
• Better error handling
• Easier debugging
• Professional API design

Common fields:
• success

• message
• data
• timestamp
• errorCode (optional)

Simple Explanation
Standard API response means: “Return all responses in a common structure instead of random formats.”

Example

```
public class ApiResponse<T> {
   private boolean success;
   private String message;
   private T data;

   public ApiResponse(boolean success, String message, T data) {
      this.success = success;
      this.message = message;
      this.data = data;
   }
}

@GetMapping("/{id}")
public ResponseEntity<ApiResponse<String>> getUser(@PathVariable Long id) {
   return ResponseEntity.ok(new ApiResponse<>(true, "User fetched", "User id: " + id));
}
```

Interviewer Cross-Question
Q: Is using a response wrapper mandatory in REST APIs?
A: No, it is not mandatory, but it is a common and useful practice in real-world projects for consistency.

27) What is @ExceptionHandler?

Interview Question
What is @ExceptionHandler in Spring Boot REST APIs?

Detailed Answer
@ExceptionHandler is used to handle specific exceptions thrown during request processing.

Instead of writing try-catch blocks in every controller method, Spring allows us to define methods that catch exceptions and return appropriate responses.

It can be used:
• Inside a controller → local exception handling
• Inside @ControllerAdvice or @RestControllerAdvice → global exception handling

Why use it?
• Cleaner controller code
• Reusable exception logic
• Better API error responses
• Centralized handling of business exceptions

Simple Explanation
@ExceptionHandler means: “Catch this exception and return a proper response.”

Example

```
@RestController
@RequestMapping("/users")
public class UserController {

   @GetMapping("/{id}")
   public String getUser(@PathVariable Long id) {
      if (id != 1) {
         throw new RuntimeException("User not found");
      }
      return "User found";
   }

   @ExceptionHandler(RuntimeException.class)
   public ResponseEntity<String> handleRuntimeException(RuntimeException ex) {
      return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ex.getMessage());
   }
}
```

Interviewer Cross-Question
Q: What is the difference between @ExceptionHandler and @ControllerAdvice?
A: @ExceptionHandler handles specific exceptions, while @ControllerAdvice makes exception handling global across controllers.

28) Difference between @ControllerAdvice and @RestControllerAdvice

Interview Question
What is the difference between @ControllerAdvice and @RestControllerAdvice?

Detailed Answer
Both annotations are used for global controller-level logic in Spring.

@ControllerAdvice
• General-purpose global advice annotation
• Can be used for exception handling, global model attributes, and binder configuration
• In REST APIs, you may need @ResponseBody in handler methods

@RestControllerAdvice
• Specialized version of @ControllerAdvice
• Equivalent to @ControllerAdvice + @ResponseBody
• Methods automatically return response body data (JSON/text)

Best practice:
• For REST APIs, @RestControllerAdvice is usually preferred.

Simple Explanation
• @ControllerAdvice = global controller advice
• @RestControllerAdvice = global controller advice + response body for REST APIs

Example

```
@RestControllerAdvice
public class GlobalExceptionHandler {

   @ExceptionHandler(RuntimeException.class)
   public ResponseEntity<String> handleRuntimeException(RuntimeException ex) {
      return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(ex.getMessage());
   }
}
```

Interviewer Cross-Question
Q: Which one is preferred in REST APIs?
A: @RestControllerAdvice, because it automatically returns JSON/text response body.

29) Difference between @NotNull, @NotEmpty, and @NotBlank

Interview Question
What is the difference between @NotNull, @NotEmpty, and @NotBlank in validation?

Detailed Answer
These are Bean Validation annotations used to validate input fields in DTOs, but they behave differently.

@NotNull
• Checks that the value is not null
• Allows empty string "" and blank string " "

@NotEmpty
• Checks that the value is not null and not empty
• For strings, "" is not allowed
• " " (spaces only) is allowed
• For collections, empty list is not allowed

@NotBlank
• Used mainly for strings
• Checks that the value is not null, not empty, and not only whitespace

This is a very common interview question for validation.

Simple Explanation
• @NotNull = value should not be null
• @NotEmpty = value should not be null or empty
• @NotBlank = value should not be null, empty, or just spaces

Example
```
public class UserRequestDto {

   @NotBlank(message = "Name is required")
   private String name;

   @NotEmpty(message = "Roles list should not be empty")
   private List<String> roles;

   @NotNull(message = "Age is required")
   private Integer age;
}
```

Interviewer Cross-Question
Q: Which one is usually best for validating a name field?
A: @NotBlank, because it rejects null, empty string, and whitespace-only values.

30) How do you configure CORS globally in Spring Boot?

Interview Question
How do you configure CORS globally in Spring Boot REST APIs?

Detailed Answer
CORS stands for Cross-Origin Resource Sharing.

Browsers block requests from different origins unless the backend explicitly allows them.

Example:
• Frontend: http://localhost:3000
• Backend: http://localhost:8080

We can enable CORS in Spring Boot in two ways:
1. Method-level or Controller-level using @CrossOrigin
2. Global-level using WebMvcConfigurer

Global configuration is preferred when many APIs need the same CORS rules.

Benefits of global CORS config:
• Avoid repeating @CrossOrigin everywhere
• Centralized configuration
• Easier maintenance

Simple Explanation
Global CORS config means: “Allow frontend applications from another origin to call all APIs from one central place.”

Example

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {

   @Override
   public void addCorsMappings(CorsRegistry registry) {
      registry.addMapping("/**")
            .allowedOrigins("http://localhost:3000")
            .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE")
            .allowedHeaders("*");
   }
}
```

Interviewer Cross-Question
Q: Is CORS enforced by Spring Boot or browser?

A: CORS is enforced by the browser. Spring Boot only sends the headers that allow cross-origin access.